Reviewer Pack — Polymatroid Rank and SOS Refutation Degree for Monotone k-CL…

Entry fa4bcf9e9740 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 13:00:29 UTC

Polymatroid Rank and SOS Refutation Degree for Monotone k-CLIQUE

Entry ID: `fa4bcf9e9740` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 13:00:29 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): SOS Refutation Degree for CSPs

Statement:

> For any CNF formula Φ on n variables, the SOS refutation degree of Φ is Θ(rank(Φ)), where rank(Φ) is the polymatroid rank of the hypergraph formed by its clauses. For k-CLIQUE instances, rank(Φ) ≥ Ω(n^{1/2}) while rank(DNF) ≤ O(log n) for any monotone DNF formula.

Rationale (proposer's reasoning):

> Polymatroid rank captures submodular structure of constraints, which SOS hierarchies exploit via semidefinite programming. Monotone k-CLIQUE's inherent combinatorial rigidity creates a gap between DNF and polymatroid rank, suggesting SOS degree lower bounds via rank growth.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `45377bdbbac3d63d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        edges = set()
        for i in range(k):
            for j in range(i + 1, k):
                edges.add((i, j))
        return [random.sample(range(n), k) for _ in range(30)]

    def generate_dnf(n):
        clauses = []
        for _ in range(30):
            clause = random.sample(range(n), random.randint(1, n // 2))
            clauses.append(clause)
        return clauses

    def hypergraph_rank(edges):
        rank = 0
        while edges:
            edge = edges.pop()
            rank += 1
            new_edges = set()
            for e in edges:
                if not any(x in e for x in edge):
                    new_edges.add(e)
            edges = new_edges
        return rank

    def sdp_relaxation(rank):
        # Placeholder for actual SOS degree computation
        return rank * 0.5  # Simplified approximation

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
```

```
    for n in n_values:
        k_clique_edges = generate_k_clique(n, int(math.sqrt(n)))
        dnf_clauses = generate_dnf(n)

        k_clique_rank = hypergraph_rank(k_clique_edges)
        dnf_rank = hypergraph_rank(dnf_clauses)

        k_clique_sdp = sdp_relaxation(k_clique_rank)
        dnf_sdp = sdp_relaxation(dnf_rank)

        results.append({
            "n": n,
            "k-clique_rank": k_clique_rank,
            "k-clique_sdp": k_clique_sdp,
            "dnf_rank": dnf_rank,
            "dnf_sdp": dnf_sdp
        })

    metric_value = sum(r['k-clique_rank'] for r in results) / len(results)
    instances_tested = len(results)

    conjecture_holds = all(r['k-clique_rank'] >= 0.7 * math.sqrt(n) and r['dnf_rank'] <= 5 * math.
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [677, 727, 773, 821, 877, 929]  # Default list of pr

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank_k_clique = sum(r['k-clique_rank'] for r in results if 'k-clique' in r['counterexampl
    mean_rank_dnf = sum(r['dnf_rank'] for r in results if 'DNF' in r['counterexample']) / len([r f
    support_fraction_k_clique = sum(1 for r in results if 'k-clique' in r['counterexample'] and r[
    support_fraction_dnf = sum(1 for r in results if 'DNF' in r['counterexample'] and r['conjectur

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean_k-clique={mean_rank_k_clique} std_k-clique=0.0 support_frac
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r['conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c708286.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c708286.py", line 58, in run_trial
    k_clique_rank = hypergraph_rank(k_clique_edges)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c708286.py", line 43, in hypergraph_ra
    new_edges.add(e)
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to unhashable type error in hypergraph_rank function, preventing data collection | next: Fix the hypergraph_rank function to handle list edges properly (e.g., convert to tuples) and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	52496
2	preregistration	ollama_remote	qwen3:8b	0	0	24729
3	novelty	ollama_remote	qwen3:8b	0	0	21316
4	novelty	ollama_remote	qwen3:8b	0	0	15538
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18331
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11809
7	judge	ollama_remote	qwen3:8b	0	0	16776

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 160997 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/fa4bcf9e9740.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa4bcf9e9740.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fa4bcf9e9740.tar.gz` (if generated)